Transmetro

Sistema de Control de Líneas, Estaciones y Buses

Guía técnica para la defensa del proyecto

Cómo está construido, qué tecnologías usa, por qué se eligieron, cómo se publica y cómo responder las preguntas técnicas del jurado.

Contenido

1. Resumen del proyecto en un minuto
2. Stack tecnológico y por qué
3. Arquitectura general
4. Estructura del repositorio
5. Base de datos
6. Reglas de negocio
7. Seguridad
8. Tiempo real y mapa
9. API REST
10. Frontend
11. Pruebas
12. Despliegue en la nube
13. Cómo se construyó
14. Preguntas probables del jurado y respuestas
15. Limitaciones honestas y trabajo futuro
16. Hoja de repaso rápido

1. Resumen del proyecto en un minuto

Transmetro es una **aplicación web completa** para administrar el sistema de buses: líneas, estaciones, accesos, guardias, parqueos, buses, pilotos, tarjetas de pasaje, cobros, alertas de ocupación, incidentes de seguridad, finanzas y análisis de demanda. Tiene una **pantalla pública** para el ciudadano (mapa de la red, horarios, recarga de tarjeta y buzón de sugerencias), un **portal del pasajero** y un **panel del personal** con permisos según el rol.

Frase para abrir la defensa:

> "Es una aplicación web de una sola página hecha en **React**, que consume una **API REST en Node.js con Express y TypeScript**. Los datos viven en **PostgreSQL** y se acceden con el ORM **Sequelize**. Las posiciones de los buses y las alertas llegan en **tiempo real con Socket.IO**. Está publicada en **Render** con la base de datos en **Neon**, y tiene **103 pruebas automáticas** con Jest y Supertest."

Dato	Valor
Tipo de aplicación	SPA (Single Page Application) + API REST + WebSockets
Lenguaje	TypeScript en backend y frontend (un solo lenguaje en todo el proyecto)
Base de datos	PostgreSQL 16 (local en Docker, en la nube en Neon)
Tablas	20 (municipalidad, linea, estacion, bus, tarjeta_acceso, transaccion, usuario, etc.)
Roles	admin, municipal, operador, guardia, pasajero
Pruebas	103 casos automáticos (unitarias + integración), cada uno con su ID del plan de pruebas (CP-xx)
Tamaño	Unas 4,800 líneas de TypeScript y 740 de SQL
Publicación	Render (servidor web) + Neon (PostgreSQL), plan gratuito

2. Stack tecnológico y por qué

Backend (servidor)

Tecnología	Para qué se usa	Por qué se eligió
Node.js 22	Entorno que ejecuta el servidor	Mismo lenguaje que el frontend; muy bueno para muchas conexiones simultáneas (E/S no bloqueante), ideal para tiempo real.
TypeScript	Lenguaje del código	JavaScript con tipos: detecta errores antes de ejecutar y documenta la forma de los datos.
Express 5	Framework HTTP: rutas, middlewares	Estándar de la industria, simple y minimalista. La versión 5 captura los errores de funciones async sin try/catch.
Sequelize 6	ORM: modelos, consultas, transacciones	Evita escribir SQL repetitivo, protege contra inyección SQL (consultas parametrizadas) y maneja transacciones y bloqueos.
PostgreSQL	Base de datos relacional	ACID, llaves foráneas, CHECK, triggers, buen soporte de fechas y zonas horarias. Gratis y de código abierto.
Socket.IO	Tiempo real (WebSockets)	Envía posición de buses, alertas y pánico al instante, sin que el navegador tenga que preguntar cada segundo.
Zod	Validación de datos de entrada	Valida cada petición y devuelve errores claros por campo (400). También valida las variables de entorno.

Tecnología	Para qué se usa	Por qué se eligió
jsonwebtoken (JWT)	Sesiones	Token firmado, sin guardar sesiones en el servidor; expira en 12 horas.
bcryptjs	Contraseñas	Hash lento con sal (10 rondas): aunque roben la base no se obtienen las contraseñas.
Helmet	Cabeceras de seguridad HTTP	Content-Security-Policy, anti-clickjacking, no-sniff, etc. con una línea.
node:crypto	Cifrado de DPI y NIT	AES-256-GCM, incluido en Node (sin librerías externas).
tsx	Ejecutar TypeScript directo	Evita un paso de compilación en el servidor.

Frontend (navegador)

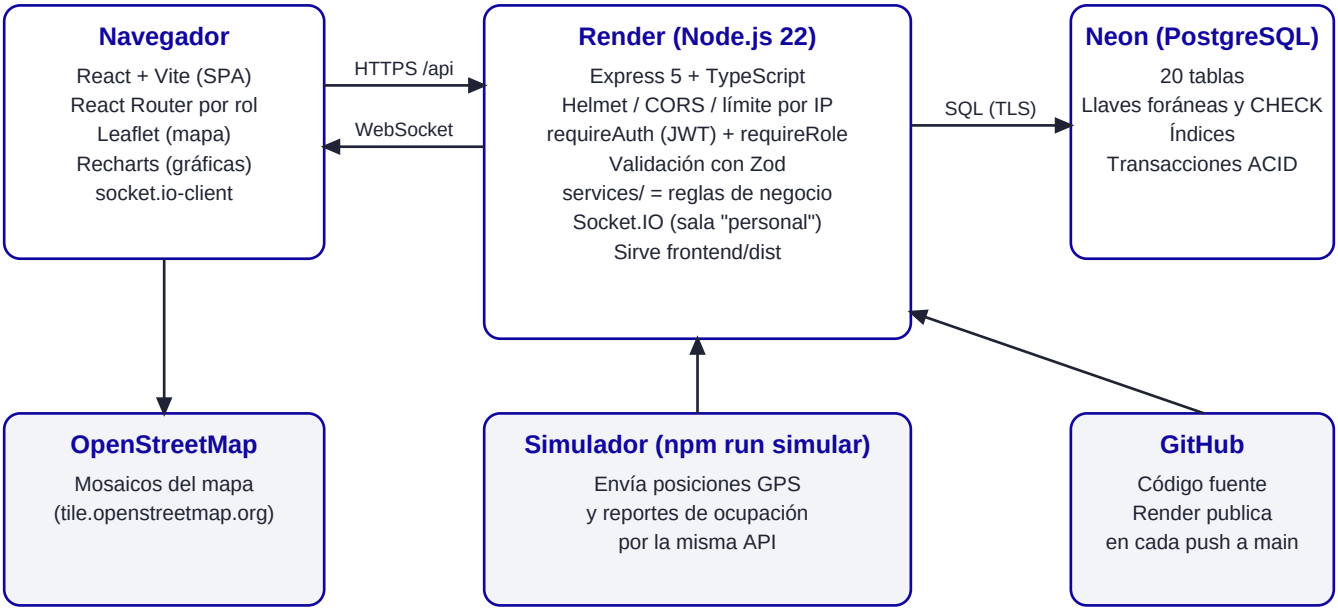
Tecnología	Para qué se usa	Por qué se eligió
React 18	Interfaz por componentes	La librería de UI más usada; componentes reutilizables y estado reactivo.
Vite 5	Servidor de desarrollo y empaquetado	Arranque instantáneo y build optimizado. Su proxy evita problemas de CORS en desarrollo.
React Router 6	Navegación entre pantallas	Rutas distintas según el rol del usuario sin recargar la página.
Tailwind CSS 3	Estilos	Clases utilitarias: diseño consistente y responsive rápido, con la paleta institucional.
Leaflet + OpenStreetMap	Mapas	Librería de mapas libre y gratuita (no requiere llave como Google Maps).
Recharts	Gráficas	Gráficas de finanzas y demanda declaradas como componentes de React.
socket.io-client	Recibir eventos en vivo	Contraparte del servidor Socket.IO.

Herramientas

Herramienta	Uso
Docker / docker-compose	Levanta PostgreSQL 16 y pgAdmin en local con un solo comando.
Jest + Supertest	Pruebas unitarias y de integración del backend.
Postman	Colección de la API (carpeta pruebas/) para probar endpoints a mano.
Git + GitHub	Control de versiones; Render descarga el código desde GitHub.
Render + Neon	Publicación gratuita: servidor web y base PostgreSQL administrada.

3. Arquitectura general

Es una arquitectura **cliente-servidor en tres capas**: presentación (React en el navegador), lógica (API Express) y datos (PostgreSQL). En producción **un solo servicio** entrega la página web y la API desde la misma dirección, así que no hay problemas de CORS y solo se paga y se mantiene un servidor.



Recorrido de una petición (ejemplo: cobrar un pasaje)

- El operador pulsa "Cobrar" en React. `api.ts` hace `POST /api/pagos` con el token JWT en la cabecera `Authorization: Bearer`.
- **Helmet** agrega cabeceras de seguridad y **express.json** lee el cuerpo (máximo 200 KB).
- **requireAuth** verifica la firma del token y recarga el usuario desde la base: si lo desactivaron, se rechaza en ese momento.
- **requireRole("admin", "operador")** confirma que el rol tiene permiso; si no, responde 403.
- **Zod** valida los datos; si algo falta responde 400 indicando el campo.
- El servicio `pagos.ts` abre una **transacción**, bloquea la fila de la tarjeta (`SELECT ... FOR UPDATE`), revisa el saldo, lo descuenta y registra la transacción. Todo se guarda junto o nada se guarda.
- Si ocurre un error, el **errorHandler** central lo traduce a un código HTTP con mensaje en español (404, 409, 500...).

4. Estructura del repositorio

Es un **monorepo**: backend y frontend viven en el mismo repositorio y se publican juntos.

```
transmetro/
|-- package.json          build: compila el frontend e instala el backend (lo usa Render)
|-- render.yaml           configuración del servicio en Render (Blueprint)
|-- docker-compose.yml    PostgreSQL 16 + pgAdmin para desarrollo local
|-- DESPLIEGUE.md         guía de publicación paso a paso
|-- database/             scripts SQL: esquema, triggers, datos demo, consultas de reportes
|-- pruebas/              colección de Postman
|-- backend/
|   |-- src/
|   |   |-- server.ts     arranque: conecta a la BD, sincroniza tablas, AUTO_SEED, Socket.IO
|   |   |-- app.ts        Express: Helmet, CORS, JSON, rutas /api, sirve el frontend
|   |   |-- db.ts         modelos Sequelize (20 tablas) y sus relaciones
|   |   |-- realtime.ts   Socket.IO con autenticación por token
|   |   |-- config/env.ts variables de entorno validadas con Zod
|   |   |-- middleware/   auth.ts (JWT + roles), limite.ts (límite de intentos por IP)
```

```
|   |   |-- lib/          http.ts (errores), cifrado.ts (AES-256-GCM)
|   |   |-- routes/       index.ts (endpoints), crud.ts (CRUD genérico reutilizable)
|   |   |-- services/     reglas de negocio: buses, lineas, pagos, capacidad, demanda, finanzas...
|   |   |-- seed.ts       datos de demostración
|   |   `-- scripts/simulador.ts   buses en movimiento para la demo
|   `-- tests/            unit/ e integration/ (Jest + Supertest)
`-- frontend/
    |-- vite.config.ts    proxy /api y /socket.io hacia el puerto 4000 en desarrollo
    `-- src/
        |-- App.tsx       rutas según el rol
        |-- api.ts        cliente HTTP con el token
        |-- auth.tsx      contexto de sesión
        |-- socket.ts     conexión en tiempo real
        |-- components/   Layout, formularios, tablas reutilizables
        `-- pages/        Inicio, Dashboard, Mapa, Buses, Cobros, Finanzas, Demanda...
```

Separación por capas en el backend: las **rutas** solo reciben y validan; la **lógica** vive en `services/`; el **acceso a datos** está en `db.ts`. Así las reglas se pueden probar por separado y no se repiten.

5. Base de datos

PostgreSQL con 20 tablas. Se definen dos veces de forma equivalente: en `database/01_esquema.sql` (el script entregado, para revisión y pgAdmin) y como modelos de Sequelize en `backend/src/db.ts`, que es lo que usa la aplicación para crear las tablas automáticamente.

Grupo	Tablas
Red	municipalidad, linea, estacion, linea_estacion (orden y distancia de cada parada), acceso, guardia
Flota	bus, parqueo, piloto, operador, auditoria_bus (historial de cambios de línea/parqueo)
Pagos	tarjeta_acceso, transaccion (pasajes y recargas)
Operación	alerta (sobrecupo, espera extra, refuerzo de flota), incidente_seguridad
Usuarios	usuario (rol, estación o municipalidad asignada, DPI/NIT cifrados)
Gestión	movimiento_financiero, sugerencia, horario_linea, horario_estacion

Cómo protege los datos la base misma

- **Llaves foráneas**: no puede existir un bus en un parqueo que no existe, ni una transacción de una tarjeta inexistente.
- **CHECK**: `saldo >= 0` (nunca saldo negativo), `monto > 0`, `capacidad_maxima > 0`, roles válidos, `hora_inicio < hora_fin`.
- **NOT NULL en bus.id_parqueo**: un bus nunca queda sin parqueo (requerimiento 7).
- **UNIQUE**: email de usuario, número de tarjeta, placa, referencia de transacción (evita cobros duplicados), huella del DPI.
- **Índices** en las columnas más consultadas: bus por línea, transacciones por fecha y línea, alertas por estado.
- **Triggers** (`02_reglas_triggers.sql`): validan el rango de buses por línea y recalculan la distancia total cuando cambian las paradas.

Cambios respecto al diagrama ER original

- Se agregaron `usuario` (roles) y `auditoria_bus` (historial, RNF-6).

- Latitud y longitud en estación y bus (mapa y GPS); `referencia` en transacción (idempotencia); `atendida` en alerta.
- Se usó **PostgreSQL en lugar de MySQL**, como definía el documento de stack.

6. Reglas de negocio

Las reglas están en `backend/src/services/`, en un solo lugar, y varias se refuerzan también en la base de datos (doble capa).

Regla	Cómo se cumple	Dónde
Una línea tiene entre N y 2N buses (N = número de estaciones)	Se valida al asignar o quitar un bus, dentro de una transacción con bloqueo de la línea para que dos operadores no pasen el límite al mismo tiempo.	buses.ts, lineas.ts, trigger
Un bus pertenece a una sola línea o a ninguna	Columna id_linea única por bus (puede ser NULL).	db.ts
Un bus nunca queda sin parqueo	id_parqueo NOT NULL; cambiarParqueo mueve, nunca deja vacío.	db.ts, buses.ts
Distancia de la línea = suma de tramos	Se recalcula al guardar el recorrido.	lineas.ts, trigger
Alerta de sobrecupo	Pasajeros en espera >= 150 % de la capacidad del bus. Crea alerta y la envía en vivo.	capacidad.ts
Espera adicional	Si el bus va a menos del 25 % de ocupación, esperar 5 minutos más.	capacidad.ts
Cobro atómico e idempotente	Transacción + bloqueo de la tarjeta; si llega la misma referencia dos veces no se cobra dos veces.	pagos.ts
Mínimo un guardia por acceso	El reporte de estaciones marca cumple_guardia_minima.	reportes.ts
Historial de asignaciones	Cada cambio de línea o parqueo se registra en auditoria_bus.	buses.ts
Demanda	Línea crítica si su carga por bus es 1.5 veces el promedio o tiene 5 alertas de sobrecupo en 7 días; recomienda buses extra sin pasar de 2N.	demanda.ts

7. Seguridad

Amenaza	Defensa implementada
Robo de contraseñas	Se guardan con **bcrypt** (hash con sal, 10 rondas). Nunca en texto plano.
Acceso sin permiso	**JWT** firmado con JWT_SECRET (expira en 12 h) + **control de acceso por roles** (RBAC) en cada ruta. El usuario se recarga en cada petición, así una cuenta desactivada pierde acceso de inmediato.
Ver datos de otra municipalidad	El personal municipal solo consulta su municipalidad (filtro automático en las consultas).
Fuerza bruta en el login	**Límite por IP**: 30 intentos cada 10 minutos en login y registro, 15 en la recarga pública y 5 en sugerencias. Mismo mensaje para "usuario no existe" y "contraseña mala".

Amenaza	Defensa implementada
Inyección SQL	Sequelize usa **consultas parametrizadas**; los SQL manuales usan replacements, nunca concatenación.
Datos inválidos o maliciosos	Todo lo que entra se valida con **Zod** (tipos, rangos, formatos). Cuerpo máximo 200 KB.
XSS y clickjacking	**Helmet** con Content-Security-Policy: solo scripts propios, imágenes propias y de OpenStreetMap. React escapa el texto por defecto.
Datos personales (DPI, NIT)	Cifrados en la base con **AES-256-GCM** (IV aleatorio por dato). Para detectar duplicados se guarda una huella **HMAC-SHA256**, no el dato. En pantalla se muestran enmascarados (••••1234).
Secretos en el código	Van en variables de entorno. En producción el servidor **se niega a arrancar** sin JWT_SECRET y DATOS_CLAVE propios. .env está en .gitignore.
Alertas a quien no debe	Socket.IO exige token; solo el personal entra a la sala "personal". Un pasajero nunca recibe alertas ni pánico.
Pagos bancarios	La pasarela está **simulada** detrás de una interfaz; nunca se reciben datos de tarjetas bancarias.

8. Tiempo real y mapa

Socket.IO mantiene una conexión abierta (WebSocket) entre el navegador del personal y el servidor. Cuando pasa algo, el servidor lo **empuja** a todos al instante, en lugar de que cada navegador pregunte cada pocos segundos (polling).

Evento	Cuándo se emite
bus:posicion	Llega una posición GPS (POST /api/buses/:id/posicion). El mapa mueve el marcador.
alerta	Sobrecupo, espera extra o refuerzo de flota.
incidente	Un guardia u operador reporta un incidente.
panico	Botón de pánico.

Mapa: Leaflet dibuja los mosaicos de OpenStreetMap, las líneas (polilíneas de colores) y las estaciones. Las posiciones de demostración las genera el **simulador** (`npm run simular`): interpola la posición de cada bus entre estaciones, ida y vuelta, y cada 30 segundos reporta ocupaciones que disparan alertas. Usa la misma API que usaría un GPS real, así que reemplazarlo por dispositivos reales no requiere cambiar el servidor.

> **Si preguntan por el error "Access blocked" del mapa:** OpenStreetMap exige que cada pedido de mosaico lleve la cabecera Referer para identificar la aplicación. Helmet la quitaba por defecto (política no-referrer). Se corrigió con `Referrer-Policy: strict-origin-when-cross-origin`, que envía solo el dominio, sin rutas ni datos.

9. API REST

Todas las rutas empiezan con `/api`, usan JSON y el token en `Authorization: Bearer <token>`. Los catálogos (municipalidades, líneas, estaciones, accesos, guardias, operadores, parqueos, buses, pilotos, tarjetas, usuarios) comparten un **CRUD genérico** (`routes/crud.ts`): una sola función crea GET, POST, PUT y DELETE con validación y permisos, lo que evita código repetido.

Área	Endpoints principales
Autenticación	POST /auth/login · POST /auth/registro · GET /auth/me
Público (sin sesión)	GET /publico/red · POST /publico/recargar · POST /publico/sugerencias
Pasajero	/mi/tarjetas · /mi/tarjetas/solicitar · /mi/tarjetas/vincular · /mi/recargar · /mi/lineas · /mi/perfil
Líneas y buses	GET /lineas/:id/detalle · GET /lineas/:id/cumplimiento · PUT /lineas/:id/estaciones · PUT /buses/:id/linea · PUT /buses/:id/parqueo · POST /buses/:id/posicion · GET /buses/:id/historial
Operación	POST /ocupacion · GET /alertas · PATCH /alertas/:id/atender · POST /pagos · POST /tarjetas/recargar · POST /incidentes · POST /incidentes/panico
Gestión	GET /finanzas/resumen · /finanzas/movimientos · GET /demanda · POST /demanda/notificar · /horarios · /sugerencias
Reportes	GET /reportes/estaciones · /reportes/lineas-buses · /reportes/accesos-por-linea (JSON o ?formato=csv) · GET /dashboard

Códigos de respuesta

Código	Significado en el sistema
200 / 201	Correcto / creado
400	Datos inválidos (Zod indica el campo)
401	Sin sesión, token vencido o cuenta desactivada
403	Con sesión pero sin permiso para esa acción
404	No existe
409	Conflicto con una regla: saldo insuficiente, línea llena, dato duplicado
429	Demasiados intentos (límite por IP)

10. Frontend

- **SPA**: se descarga una vez y luego solo intercambia JSON con la API; navegar no recarga la página.
- **Rutas por rol** (`App.tsx`): sin sesión se ve la pantalla pública; el pasajero ve "Mi tarjeta" y "Recorridos"; el personal ve el panel con las opciones de su rol.
- **Sesión**: el token se guarda en localStorage; `api.ts` lo agrega a cada petición y, si el servidor responde 401, cierra la sesión.
- **Componentes reutilizables**: `ResourcePage` arma tablas y formularios de los catálogos; `Layout` da el menú y la barra superior.
- **Identidad visual**: logo y paleta institucional (azul #10069F, verde, gris #F2F4F7), diseño responsive con Tailwind.
- **Pantallas**: Inicio pública, Dashboard, Líneas, Estaciones, Buses, Parqueos, Pilotos, Cobros, Alertas, Mapa en vivo, Reportes, Finanzas, Demanda, Horarios, Sugerencias, Mi tarjeta, Recorridos, Registro.
- En desarrollo, el **proxy de Vite** reenvía /api y /socket.io al puerto 4000, así el navegador habla con un solo origen.

11. Pruebas

103 casos automáticos con **Jest** y **Supertest**, cada uno marcado con su ID del Plan de Pruebas (CP-01, CP-02, ...).

Tipo	Qué prueba	Ejemplos
Unitarias (tests/unit)	Funciones puras, sin base de datos	Rango N a 2N, umbral de 150 %, distancias, CSV, índices de demanda y finanzas
Integración (tests/integration)	Endpoints completos contra una base de pruebas real	Login y roles, asignar bus a línea llena (409), cobro sin saldo, cobro duplicado, alertas, reportes, endpoints públicos

- Por seguridad, `tests/setup.ts` se niega a correr si la base no tiene "test" en el nombre, para no borrar datos reales.
- Comandos: `npm test` y `npm run test:cov` (cobertura).
- Además: colección de **Postman** en `pruebas/` y revisión de tipos con `tsc --noEmit`.

12. Despliegue en la nube

Pieza	Servicio	Detalle
Web + API + tiempo real	Render (Web Service, plan Free)	Un solo servicio Node. Se duerme tras 15 min sin visitas; despertar tarda cerca de 1 minuto.
Base de datos	Neon (PostgreSQL, plan Free)	0.5 GB. Conexión cifrada (sslmode=require).
Código	GitHub	Cada push a main hace que Render publique de nuevo automáticamente.

Qué hace Render en cada publicación

```
1. Descarga el código de GitHub (rama main)
2. Build:  npm run build
           -> instala y compila el frontend (vite build -> frontend/dist)
           -> instala las dependencias del backend
3. Start:  npm start  -> tsx src/server.ts
           -> conecta a Neon, crea las tablas que falten (sequelize.sync)
           -> si AUTO_SEED=true y no hay usuarios, carga los datos de demostración
           -> asegura que existan las cuentas del personal
           -> Express sirve la API en /api y la web desde frontend/dist
4. Health check:  GET /health -> {"ok": true}
```

Variables de entorno

Variable	Para qué
DATABASE_URL	Cadena de conexión a Neon
JWT_SECRET	Firma de los tokens de sesión (obligatoria en producción)
DATOS_CLAVE	Clave del cifrado AES de DPI y NIT (obligatoria en producción)
SEED_ADMIN_PASSWORD	Contraseña privada del administrador
AUTO_SEED	Carga datos de demostración si la base está vacía

Variable	Para qué
NODE_ENV / NODE_VERSION	production / 22

Por qué Render + Neon y no AWS: ambos son gratis y no piden tarjeta, así que no hay riesgo de cobro. AWS exige tarjeta y armar servidor, red y base por separado. Para producción real se podría migrar a AWS (EC2 o ECS + RDS) sin cambiar el código, porque la app solo depende de variables de entorno.

13. Cómo se construyó

- **1. Requerimientos**: se tomaron los requerimientos funcionales (RF) y no funcionales (RNF) del enunciado y se convirtieron en reglas de negocio concretas.
- **2. Modelo de datos**: diagrama entidad-relación y script SQL (`database/01_esquema.sql`), con restricciones y triggers.
- **3. Backend**: modelos Sequelize equivalentes al script, servicios con las reglas, rutas con validación y permisos, manejo central de errores.
- **4. Frontend**: pantallas en React por rol, conectadas a la API; mapa con Leaflet y gráficas con Recharts.
- **5. Tiempo real**: Socket.IO para posiciones, alertas e incidentes, más el simulador para la demostración.
- **6. Pruebas**: casos unitarios y de integración ligados al Plan de Pruebas.
- **7. Ampliaciones**: Línea 4 San José Pinula, solicitud de tarjeta con DPI/NIT cifrados, horarios editables, sugerencias, finanzas y análisis de demanda.
- **8. Publicación**: Render + Neon, con guía paso a paso (DESPLIEGUE.md).

Si preguntan por el uso de herramientas de IA, respondan con honestidad cómo se usaron en su equipo (por ejemplo, como apoyo para generar código y depurar), y muestren que entienden cada parte: este documento sirve justamente para eso.

14. Preguntas probables del jurado y respuestas

Arquitectura y stack

¿Por qué Node.js y no Java, PHP o Python?

Por usar un solo lenguaje (TypeScript) en todo el proyecto, y porque Node maneja muy bien muchas conexiones abiertas a la vez, lo que necesitamos para el tiempo real con Socket.IO. Express es el framework más usado y con más documentación.

¿Por qué TypeScript?

Los tipos detectan errores al escribir y no en producción; por ejemplo, si un servicio espera un número y le paso texto, el editor lo marca. Además sirve de documentación.

¿Qué es un ORM y por qué Sequelize?

Un ORM traduce tablas a objetos del código. Con Sequelize escribimos Bus.findByPk(id) en lugar de SQL a mano; además usa consultas parametrizadas (protege contra inyección SQL) y facilita transacciones y bloqueos. Para los reportes complejos usamos SQL directo con parámetros.

¿Por qué PostgreSQL y no MySQL o MongoDB?

Los datos son muy relacionales (líneas, estaciones, buses, tarjetas, transacciones), así que conviene una base relacional. PostgreSQL ofrece transacciones ACID, CHECK, triggers y buen manejo de zonas horarias, y Neon lo ofrece gratis. MongoDB no garantiza tan fácil la integridad entre tablas, que aquí es clave (por ejemplo, el dinero de las tarjetas).

¿Qué es una API REST?

Una interfaz donde cada recurso tiene una URL (/api/buses) y se usan los verbos HTTP: GET consulta, POST crea, PUT actualiza, DELETE elimina. Responde en JSON con códigos de estado (200, 400, 401, 403, 404, 409).

¿Qué es una SPA?

Single Page Application: el navegador descarga la aplicación una vez y luego solo pide datos en JSON. Es más rápida y se siente como una aplicación de escritorio.

¿Cómo se comunican frontend y backend?

Por HTTPS con JSON a /api, y por WebSocket (Socket.IO) para los eventos en vivo. En producción ambos salen del mismo servidor y la misma dirección.

Base de datos

¿Cómo garantizan que no se cobre dos veces ni quede saldo negativo?

El cobro es una transacción ACID: se bloquea la fila de la tarjeta (SELECT FOR UPDATE), se revisa el saldo, se descuenta y se registra todo junto; si algo falla, se revierte. Además, cada cobro puede llevar una referencia única: si llega repetida, se devuelve la transacción anterior sin cobrar. Y la base tiene CHECK saldo >= 0 como última barrera.

¿Qué pasa si dos operadores asignan un bus a la misma línea al mismo tiempo?

La validación del límite 2N se hace dentro de una transacción que bloquea la fila de la línea, así la segunda petición espera a la primera y ve el conteo actualizado. No se puede pasar del límite.

¿Qué es ACID?

Atomicidad (todo o nada), Consistencia (se respetan las reglas), Aislamiento (las transacciones simultáneas no se mezclan) y Durabilidad (lo confirmado no se pierde).

¿Para qué sirven los índices?

Aceleran búsquedas frecuentes, como transacciones por fecha o buses por línea, a cambio de un poco más de espacio y escritura algo más lenta.

¿Los triggers del script SQL se ejecutan en la app publicada?

Las tablas de la app las crea Sequelize, que no instala triggers. Por eso las mismas reglas están en los servicios del backend, dentro de transacciones, y probadas con pruebas automáticas. Los triggers del script SQL son una segunda capa cuando la base se crea con los scripts de la carpeta database/.

¿Cómo manejan los cambios de estructura (migraciones)?

En desarrollo, sequelize.sync con alter ajusta las tablas. En producción solo se crean las que faltan; para cambios de estructura se vuelve a cargar con npm run seed. Lo correcto a futuro son migraciones versionadas; está en el trabajo pendiente.

Seguridad

¿Cómo funciona el inicio de sesión?

El usuario manda correo y contraseña; el servidor busca el usuario y compara la contraseña con su hash bcrypt. Si coincide, devuelve un JWT firmado con el id del usuario, válido 12 horas. El navegador lo manda en cada petición y el servidor verifica la firma.

¿Qué es un JWT y por qué no sesiones en servidor?

JSON Web Token: un texto firmado digitalmente que dice "soy el usuario 5". Si alguien lo altera, la firma no coincide. No hay que guardar sesiones en el servidor, lo que simplifica escalar. Como recargamos el usuario en cada petición, desactivar una cuenta tiene efecto inmediato.

¿Qué es bcrypt y por qué no guardar la contraseña cifrada?

bcrypt es un hash de una sola vía, lento a propósito y con sal: no se puede revertir y probar millones de contraseñas cuesta mucho. El cifrado sí se puede revertir con la clave, por eso para contraseñas se usa hash, y cifrado solo para datos que hay que volver a mostrar (DPI, NIT).

¿Cómo protegen el DPI y el NIT?

Con AES-256-GCM, un cifrado moderno que además detecta si el dato fue alterado. Cada dato lleva un IV aleatorio. Para saber si un DPI ya está registrado sin descifrar todos, guardamos una huella HMAC. En pantalla solo se ven los últimos 4 dígitos.

¿Cómo funcionan los roles?

Cada ruta declara qué roles la pueden usar con requireRole(...). Por ejemplo, finanzas solo admin; cobros admin y operador; pánico admin, operador y guardia. El personal municipal además solo ve datos de su municipalidad.

¿Cómo evitan ataques de fuerza bruta?

Un limitador por IP permite 30 intentos cada 10 minutos en login y registro (15 en la recarga pública); después responde 429. Detrás del proxy de Render se usa la IP real del visitante (trust proxy).

¿Qué es CORS y Helmet?

CORS es la regla del navegador que impide que otra página use nuestra API; solo se permite nuestro origen. Helmet agrega cabeceras de seguridad, como la Content-Security-Policy, que solo deja cargar scripts propios.

Tiempo real y operación

¿Cómo se mueven los buses en el mapa?

Cada bus reporta su posición con POST /api/buses/:id/posicion. El servidor la guarda y la emite por Socket.IO con el evento bus:posicion; el mapa mueve el marcador. En la demo, el simulador hace esos reportes; en la realidad lo haría un GPS en el bus.

¿Qué es un WebSocket y por qué no recargar cada segundo?

Es una conexión que se queda abierta y por la que el servidor puede enviar datos cuando quiera. Preguntar cada segundo (polling) gastaría muchas peticiones y aun así llegaría tarde.

¿Cómo se generan las alertas de sobrecupo?

El operador reporta pasajeros en espera y a bordo. Si los que esperan son 150 % o más de la capacidad del bus, se crea una alerta SOBRECUPO; si el bus va a menos del 25 %, se recomienda esperar 5 minutos. La alerta se guarda y se envía al instante al personal.

¿Los datos de finanzas y demanda son reales?

No, son datos de demostración generados por el seed (120 días de pasajes y recargas y 5 meses de movimientos) para que los tableros tengan contenido. Los cálculos sí son los reales y funcionan igual con datos reales.

Despliegue y calidad

¿Dónde está publicado y cuánto cuesta?

En Render (servidor) y Neon (base de datos), ambos en plan gratuito y sin tarjeta, así que no hay costo. Cada cambio que subimos a la rama main en GitHub se publica solo.

¿Por qué tarda en abrir la primera vez?

El plan gratuito de Render duerme el servicio tras 15 minutos sin visitas; el primer acceso lo despierta en cerca de un minuto. En un plan pagado está siempre activo.

¿Cómo escalaría a todo el Transmetro real?

Más instancias del servidor detrás de un balanceador; el límite por IP y Socket.IO pasarían a usar Redis para compartir estado entre instancias; la base con réplicas de lectura; GPS reales enviando posiciones a la misma API; y migraciones versionadas.

¿Cómo probaron que funciona?

Con 103 pruebas automáticas (Jest + Supertest) que levantan la API y la prueban contra una base real de pruebas, cada una ligada a un caso del Plan de Pruebas. Además, Postman para pruebas manuales y la revisión de tipos de TypeScript.

¿Qué pasa si se cae internet en una estación?

El backend ya acepta una referencia única por cobro, así que se podrían guardar los cobros sin conexión y sincronizarlos después sin duplicarlos. La parte del navegador sin conexión (PWA) está en el trabajo pendiente.

15. Limitaciones honestas y trabajo futuro

Reconocer límites con naturalidad da confianza al jurado. Estos son los puntos conocidos:

- **Migraciones**: falta un sistema de migraciones versionadas para producción.
- **Pasarela de pago simulada**: la recarga no cobra dinero real; está aislada detrás de una interfaz para conectar un proveedor real.
- **Modo sin conexión** en la PC de estación (PWA + IndexedDB) pendiente; el backend ya lo soporta con la referencia idempotente.
- **Datos de demostración**: coordenadas, distancias, finanzas y demanda son aproximados o ficticios.
- **Escalabilidad**: el límite por IP es en memoria; con varias instancias se usaría Redis.
- **Plan gratuito**: el servidor duerme sin uso y la base tiene 0.5 GB.
- **Documentación de API**: falta Swagger/OpenAPI; hoy está en el README y en la colección de Postman.
- **CI/CD**: las pruebas se corren a mano; falta ejecutarlas con GitHub Actions en cada cambio.

16. Hoja de repaso rápido

Si preguntan...	Respuesta corta
Stack	React + Vite + Tailwind | Node + Express 5 + TypeScript | PostgreSQL + Sequelize | Socket.IO | Leaflet | Recharts
Arquitectura	Cliente-servidor en 3 capas, SPA + API REST + WebSockets, monorepo, un solo servicio en producción
Sesión	JWT de 12 h + bcrypt + roles con requireRole
Integridad del dinero	Transacción ACID + SELECT FOR UPDATE + referencia idempotente + CHECK saldo >= 0
Regla N a 2N	services/buses.ts dentro de transacción con bloqueo de la línea
Sobrecupo	>= 150 % de capacidad -> alerta; bus < 25 % -> esperar 5 min
Datos personales	AES-256-GCM + huella HMAC + enmascarado
Tiempo real	Socket.IO, sala "personal", eventos bus:posicion, alerta, incidente, panico
Pruebas	103 casos Jest + Supertest ligados al Plan de Pruebas
Nube	Render (web + API) + Neon (PostgreSQL), gratis, publica solo al hacer push a main
Validación	Zod en cada entrada; errores 400 por campo
Mapa	Leaflet + mosaicos de OpenStreetMap; simulador para la demo

Consejo final: durante la demo tengan abiertos el mapa con el simulador corriendo, el panel de alertas y una terminal con `npm test`. Mostrar que funciona responde muchas preguntas antes de que las hagan.